

Universidad de Oriente	
Ciencia de datos	Clase 06 - Ejemplo 1 - Análisis exploratorio de datos

```
In [118... import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from collections import Counter
```

A partir de un conjunto de datos sobre el rendimiento académico de 1000 estudiantes, realice la exploración, limpieza y preparación de los datos. Posteriormente, construya un modelo de regresión lineal para estimar la calificación final en función de las características académicas disponibles. Evalúe el ajuste del modelo e interprete la influencia de las variables predictoras.

Análisis descriptivo de los datos

```
In [122... df_estudiantes = pd.read_csv("student_performance_updated_1000.csv")
```

```
In [124... df_estudiantes
```

Out[124...

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrade
0	1.0	John	Male	85.0	15.0	78.0
1	2.0	Sarah	Female	90.0	20.0	85.0
2	3.0	Alex	Male	78.0	10.0	65.0
3	4.0	Michael	Male	92.0	25.0	90.0
4	5.0	Emma	Female	NaN	18.0	82.0
...	...	...	...	...	...	...
995	NaN	Kenneth Murray	Male	85.0	20.0	NaN
996	4497.0	Amy Stout	Female	91.0	NaN	86.0
997	1886.0	NaN	Male	85.0	8.0	82.0
998	7636.0	Joseph Sherman	Male	88.0	17.0	60.0
999	8021.0	Maria Walls	Female	88.0	10.0	90.0

1000 rows × 12 columns



In [126...

```
df_estudiantes.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   StudentID                            960 non-null    float64
1   Name                                  966 non-null    object
2   Gender                               952 non-null    object
3   AttendanceRate                        960 non-null    float64
4   StudyHoursPerWeek                     950 non-null    float64
5   PreviousGrade                         967 non-null    float64
6   ExtracurricularActivities             957 non-null    float64
7   ParentalSupport                       978 non-null    object
8   FinalGrade                            960 non-null    float64
9   Study Hours                           976 non-null    float64
10  Attendance (%)                         959 non-null    float64
11  Online Classes Taken                  975 non-null    object
dtypes: float64(8), object(4)
memory usage: 93.9+ KB
```

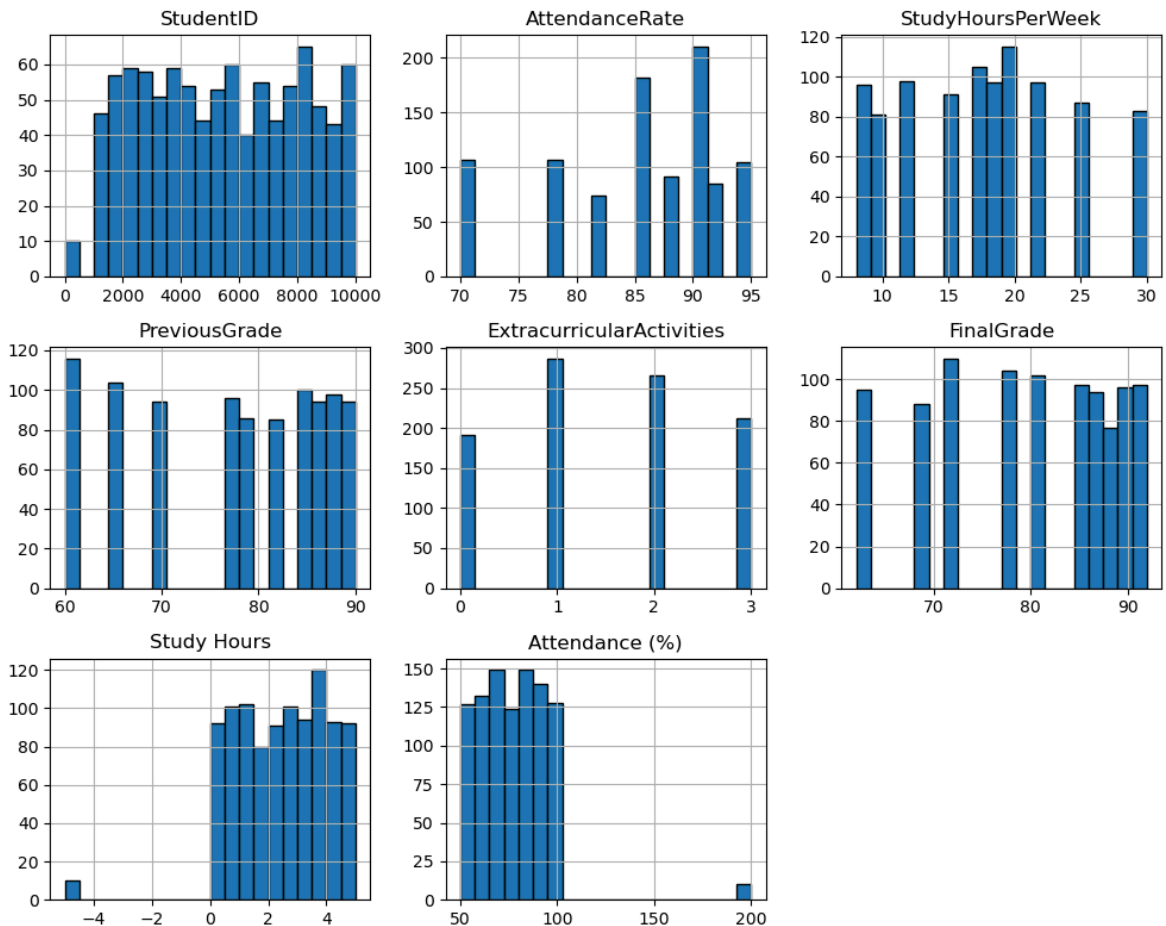
Corrección del tipo de variable

```
In [129... df_estudiantes['Gender'] = df_estudiantes['Gender'].astype('category')
df_estudiantes['ParentalSupport'] = df_estudiantes['ParentalSupport'].astype('ca
df_estudiantes['Online Classes Taken'] = df_estudiantes['Online Classes Taken'].
```

```
In [130... import matplotlib.pyplot as plt

plt.figure()
df_estudiantes.hist(figsize=(10,8),bins=20, edgecolor="black")
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>



```
In [132... df_estudiantes.isnull().sum()
```

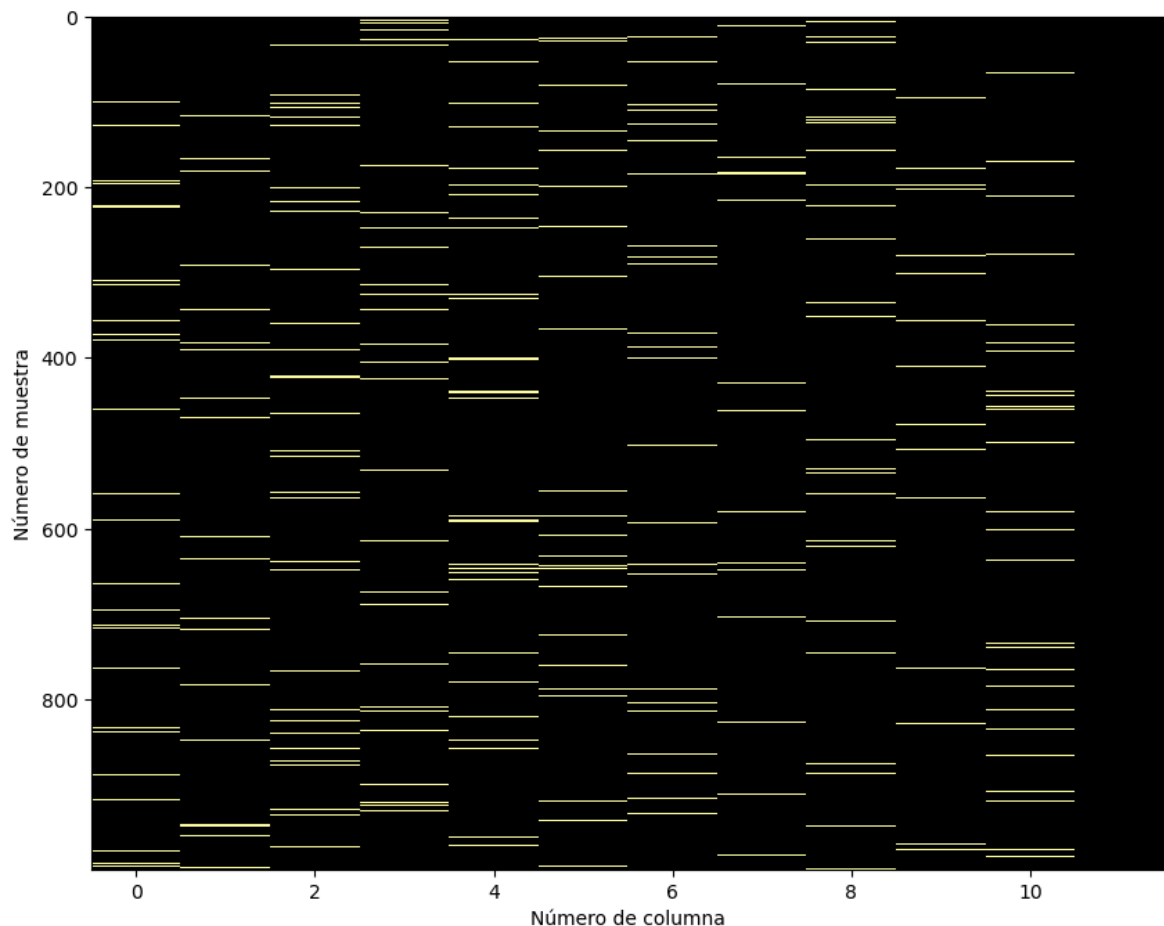
```
Out[132... StudentID          40
Name              34
Gender            48
AttendanceRate    40
StudyHoursPerWeek 50
PreviousGrade     33
ExtracurricularActivities 43
ParentalSupport   22
FinalGrade        40
Study Hours       24
Attendance (%)    41
Online Classes Taken 0
dtype: int64
```

El conjunto de datos presenta valores faltantes en casi todas sus variables. La variable más afectada es StudyHoursPerWeek, con 50 datos ausentes, seguida de Gender, con 48,

y ExtracurricularActivities, con 43. En cambio, Online Classes Taken está completa.

Representación visual de los valores nulos

```
In [137... plt.figure(figsize=(10, 8))
plt.imshow(df_estudiantes.isna(), aspect="auto", interpolation="nearest", cmap="
plt.xlabel("Número de columna")
plt.ylabel("Número de muestra");
```



Además, se observa que la variable Study Hours y Attendance (%) tiene valores incongruentes, por lo que dichos valores deben ser reemplazados con valores nulos y luego ser imputados.

```
In [140... df_estudiantes.loc[df_estudiantes["Study Hours"] < 0, "Study Hours"] = np.nan
df_estudiantes.loc[df_estudiantes["Attendance (%)"] > 100, "Attendance (%)"] = np
```

```
In [142... columnas_cuantitativas = df_estudiantes.select_dtypes(include=[np.number]).column
columnas_cuantitativas
```

```
Out[142... ['StudentID',
'AttendanceRate',
'StudyHoursPerWeek',
'PreviousGrade',
'ExtracurricularActivities',
'FinalGrade',
'Study Hours',
'Attendance (%)']
```

```
In [144... columnas_cuantitativas.remove("ExtracurricularActivities")
```

```
In [146... # Calculamos la media para cada una de las variables numéricas sin tener en cuenta
cols_mean = df_estudiantes[columnas_cuantitativas].mean()

# Sustituimos los valores NaN por la media correspondiente a cada variable
df_estudiantes[columnas_cuantitativas] = df_estudiantes[columnas_cuantitativas].
```

```
In [148... df_estudiantes.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   StudentID                            1000 non-null   float64
1   Name                                  966 non-null    object
2   Gender                               952 non-null    category
3   AttendanceRate                       1000 non-null   float64
4   StudyHoursPerWeek                   1000 non-null   float64
5   PreviousGrade                       1000 non-null   float64
6   ExtracurricularActivities           957 non-null    float64
7   ParentalSupport                     978 non-null    category
8   FinalGrade                          1000 non-null   float64
9   Study Hours                         1000 non-null   float64
10  Attendance (%)                      1000 non-null   float64
11  Online Classes Taken                1000 non-null   bool
dtypes: bool(1), category(2), float64(8), object(1)
memory usage: 73.6+ KB
```

```
In [150... cols_categoricas = ["Gender", "ParentalSupport", "ExtracurricularActivities", "C

for vars in cols_categoricas:
    df_estudiantes[vars] = df_estudiantes[vars].fillna(df_estudiantes[vars].mode
```

```
In [152... df_estudiantes.info()
```

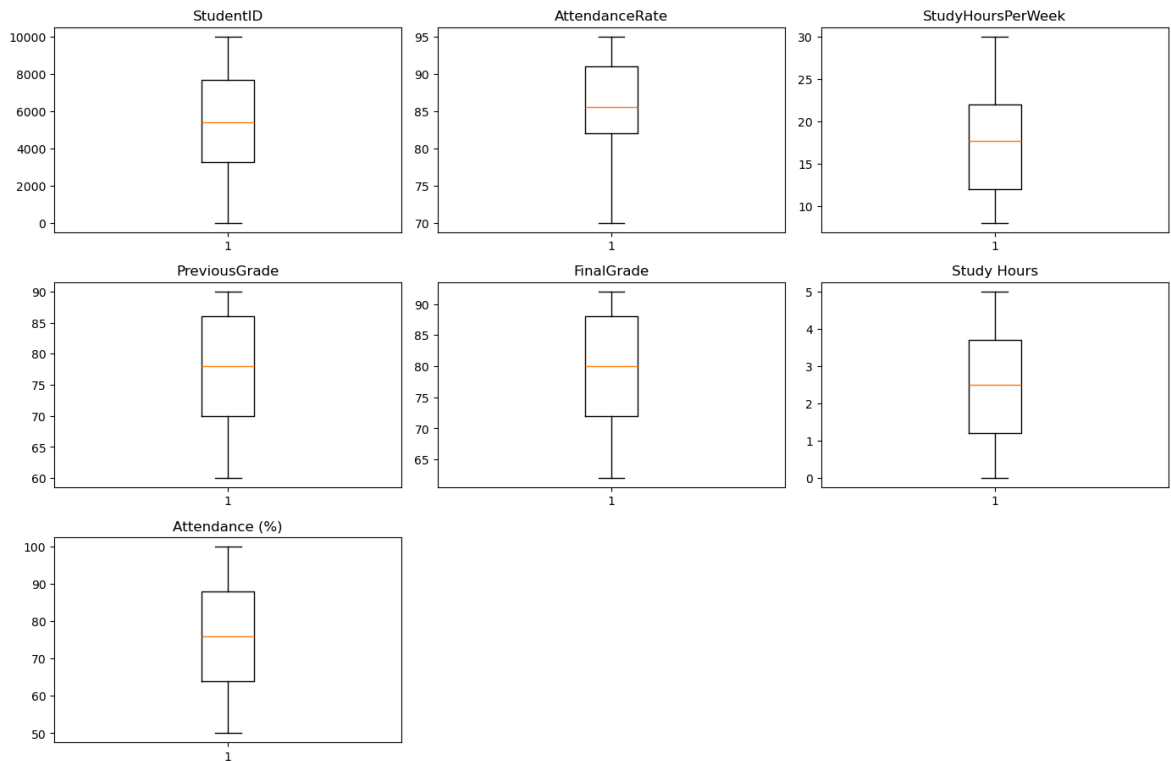
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   StudentID                            1000 non-null   float64
1   Name                                  966 non-null    object
2   Gender                               1000 non-null   category
3   AttendanceRate                       1000 non-null   float64
4   StudyHoursPerWeek                   1000 non-null   float64
5   PreviousGrade                       1000 non-null   float64
6   ExtracurricularActivities           1000 non-null   float64
7   ParentalSupport                     1000 non-null   category
8   FinalGrade                          1000 non-null   float64
9   Study Hours                         1000 non-null   float64
10  Attendance (%)                      1000 non-null   float64
11  Online Classes Taken                1000 non-null   bool
dtypes: bool(1), category(2), float64(8), object(1)
memory usage: 73.6+ KB
```

```
In [154... ## Análisis de valores atípicos
```

```
In [156... n = len(columnas_cuantitativas)

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(columnas_cuantitativas, 1):
    plt.subplot(n, 3, idx)
    plt.boxplot(df_estudiantes[var])
    plt.title(var)

plt.tight_layout()
plt.show()
```



```
In [157... def outliers_IQR(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

def outliers_desv(x):
    mu = x.mean()
    sigma = x.std()
    li = mu - 3 * sigma
    ls = mu + 3 * sigma
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

def outliers_z(df, n, features):
    outlier_list = []

    for column in features:
        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        z_score = abs( (df[column] - data_media) / data_std )
```

```

        outlier_list_column = df[z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[z_score > umbral]

    return df1.shape[0]

from scipy.stats import median_abs_deviation

def z_scoremod_method(df,n,var):
    outlier_list = []

    for column in var:

        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        MAD = median_abs_deviation

        mod_z_score = abs(0.6745*(df[column] - data_media)/MAD(df[column]))
        outlier_list_column = df[mod_z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[mod_z_score > umbral]

    return df1.shape[0]

```

```

In [158... lista_IQR = [outliers_IQR(df_estudiantes[vars]) for vars in columnas_cuantitativ
lista_desv = [outliers_desv(df_estudiantes[vars]) for vars in columnas_cuantitativ
lista_z = [outliers_z(df_estudiantes,1,[vars]) for vars in columnas_cuantitativa
lista_modz = [z_scoremod_method(df_estudiantes,1,[vars]) for vars in columnas_cu

total_outliers = pd.DataFrame({"Variables": columnas_cuantitativas, "Método IQR"
                                "Método desviación": lista_desv, "Método puntuaci
                                "Método puntuación z modificado": lista_modz})

```

```

In [162... total_outliers

```

```

Out[162...

```

	Variables	Método IQR	Método desviación	Método puntuación Z	Método puntuación z modificado
0	StudentID	0	0	0	0
1	AttendanceRate	0	0	0	0
2	StudyHoursPerWeek	0	0	0	0
3	PreviousGrade	0	0	0	0
4	FinalGrade	0	0	0	0
5	Study Hours	0	0	0	0
6	Attendance (%)	0	0	0	0

Los resultados muestran que ninguna de las variables numéricas presenta valores atípicos, ya que los métodos IQR, desviación estándar, puntuación Z y puntuación Z modificada identificaron cero observaciones extremas en todas las variables.

```
In [165... from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=2026)

anomalias = iso.fit_predict(df_estudiantes[columnas_cuantitativas])
puntaje_anomalias = iso.decision_function(df_estudiantes[columnas_cuantitativas])

df_estudiantes["Anomalías"] = anomalias
df_estudiantes['Puntaje de anomalía'] = puntaje_anomalias
```

```
In [166... df_estudiantes
```

```
Out[166...
```

	StudentID	Name	Gender	AttendanceRate	StudyHoursPerWeek	PreviousGrac
0	1.000000	John	Male	85.000000	15.000000	78.000000
1	2.000000	Sarah	Female	90.000000	20.000000	85.000000
2	3.000000	Alex	Male	78.000000	10.000000	65.000000
3	4.000000	Michael	Male	92.000000	25.000000	90.000000
4	5.000000	Emma	Female	85.510417	18.000000	82.000000
...	...	...	...	...	...	...
995	5416.019792	Kenneth Murray	Male	85.000000	20.000000	77.598750
996	4497.000000	Amy Stout	Female	91.000000	17.630526	86.000000
997	1886.000000	NaN	Male	85.000000	8.000000	82.000000
998	7636.000000	Joseph Sherman	Male	88.000000	17.000000	60.000000
999	8021.000000	Maria Walls	Female	88.000000	10.000000	90.000000

1000 rows × 14 columns

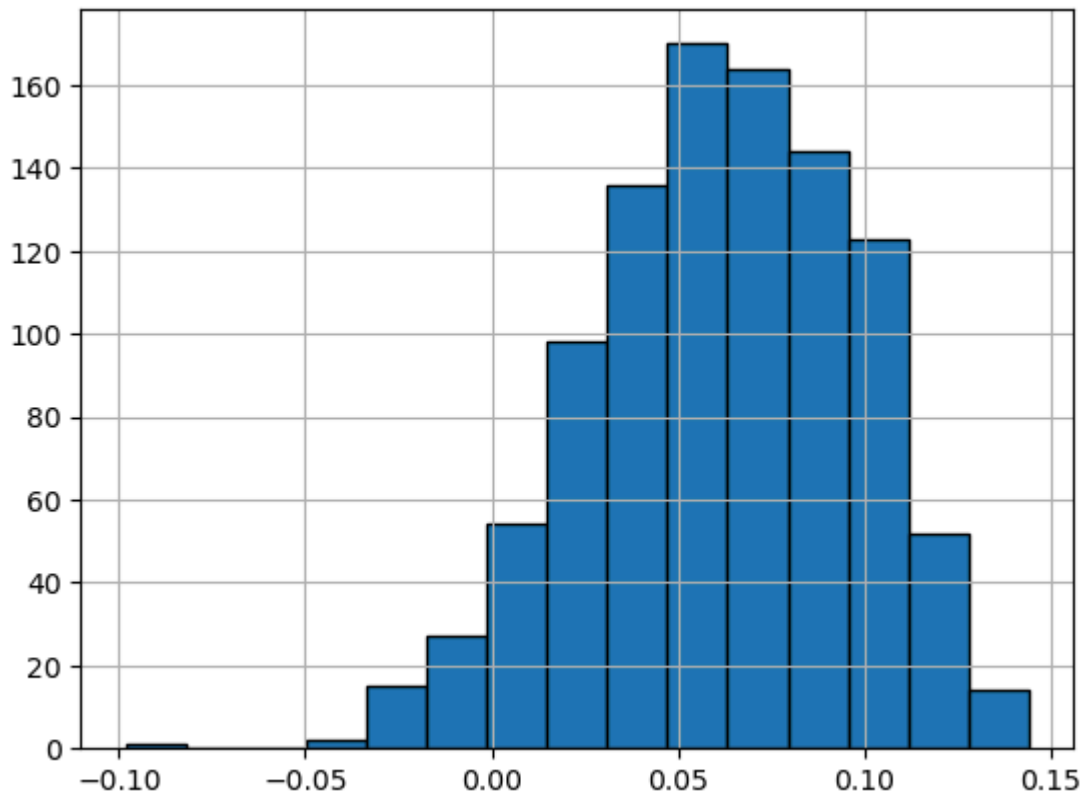


```
In [167... df_estudiantes['Anomalías'].value_counts()
```

```
Out[167... Anomalías
1      950
-1      50
Name: count, dtype: int64
```

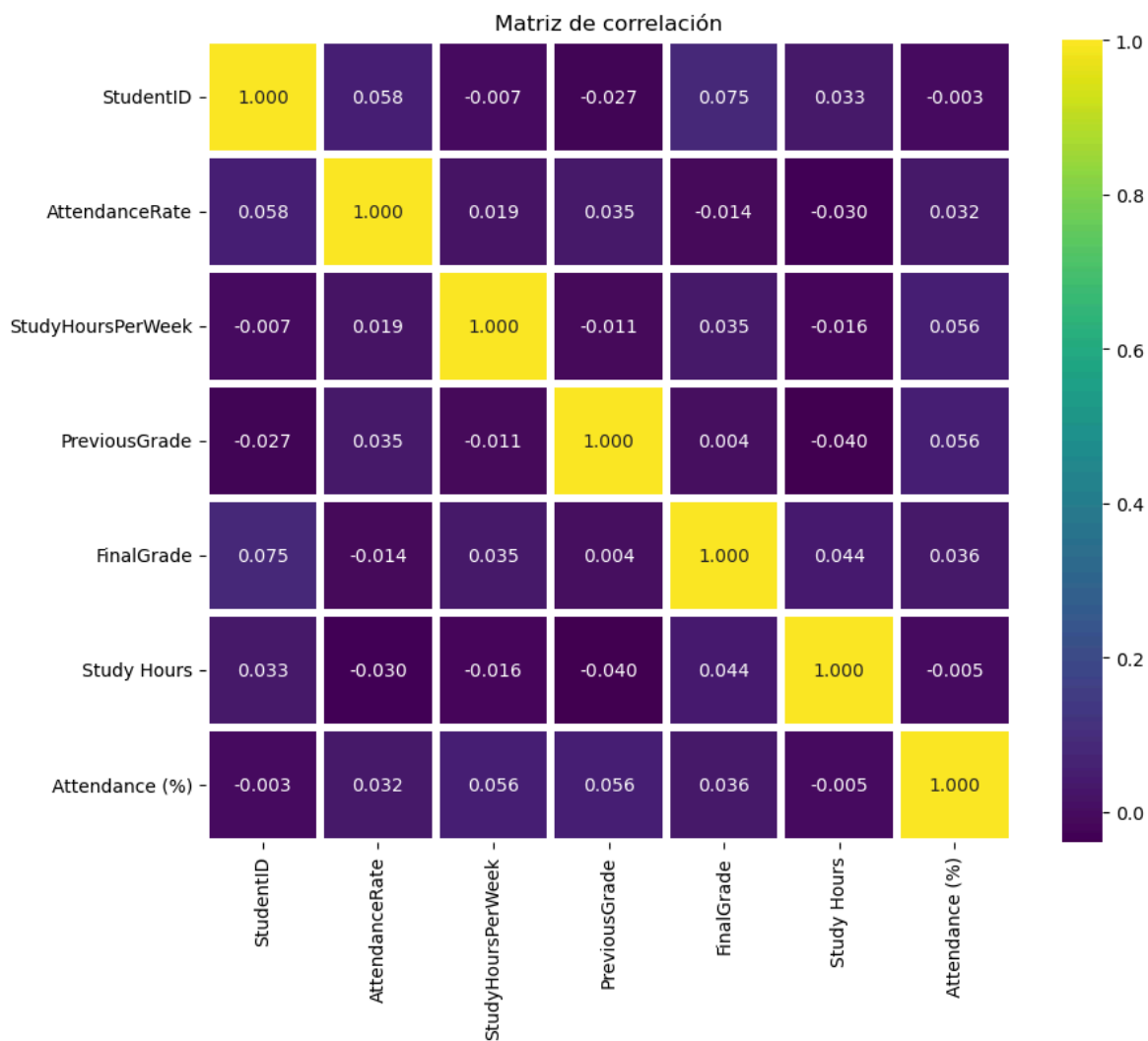
El modelo de detección de anomalías identificó 950 observaciones normales, equivalentes al 95% del conjunto de datos, y 50 observaciones anómalas, correspondientes al 5%.


```
In [172... plt.figure()  
df_estudiantes['Puntaje de anomalía'].hist(bins=15,edgecolor="black")  
plt.show()
```



Análisis de correlación entre variables

```
In [175... import seaborn as sns  
  
plt.figure(figsize=(10, 8))  
sns.heatmap(df_estudiantes[columnas_cuantitativas].corr(), annot=True, fmt='%.3f')  
  
plt.title('Matriz de correlación')  
plt.show()
```

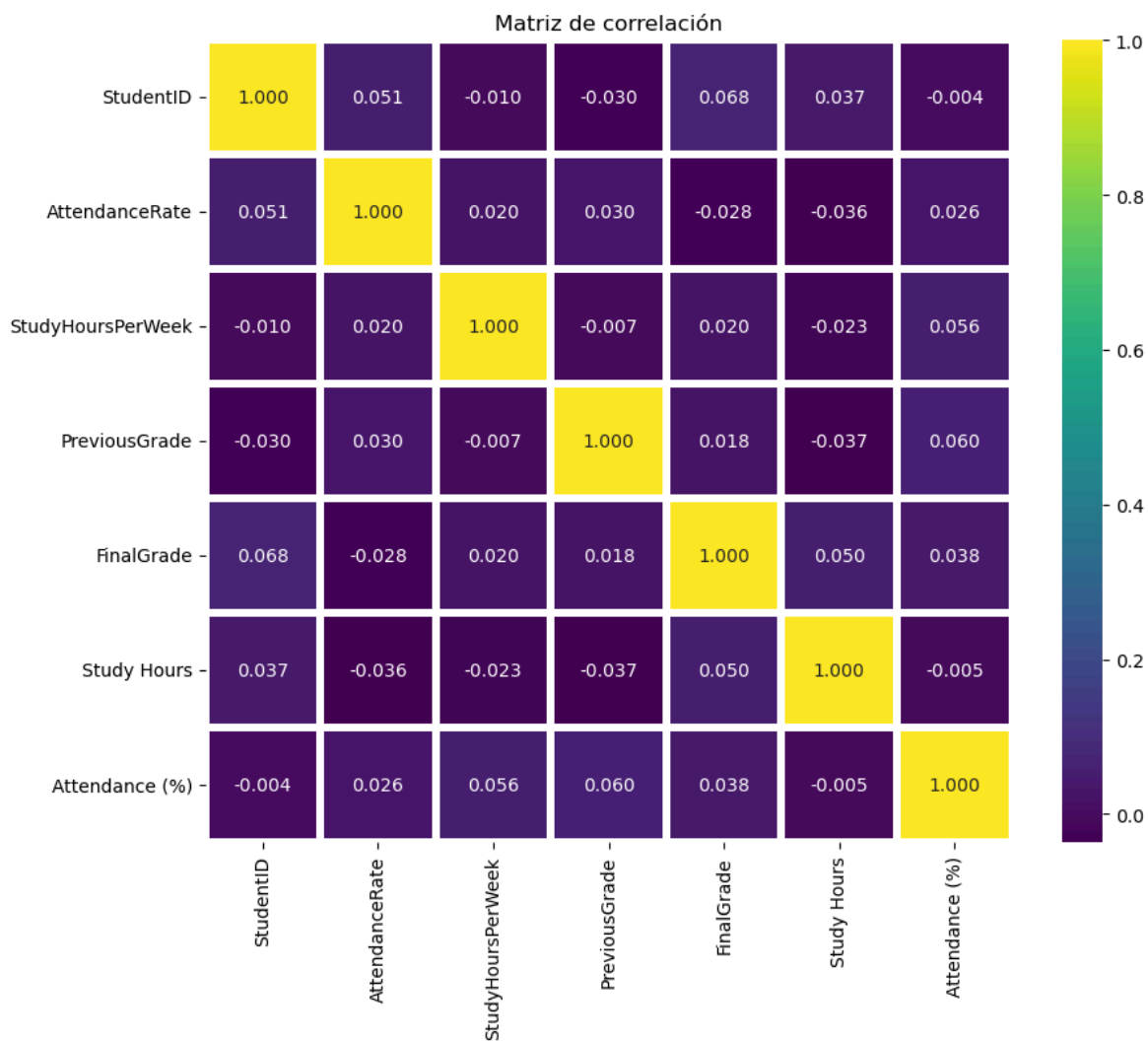


In [176...

```
import seaborn as sns

plt.figure(figsize=(10, 8))
sns.heatmap(df_estudiantes[columnas_cuantitativas].corr(method="spearman"), anno

plt.title('Matriz de correlación')
plt.show()
```



In [178...

```
import seaborn as sns

plt.figure(figsize=(10, 8))
sns.heatmap(df_estudiantes[columnas_cuantitativas].corr(method="kendall"), annot=True)

plt.title('Matriz de correlación')
plt.show()
```



La matriz de correlación evidencia que no existen relaciones lineales significativas entre las variables numéricas del conjunto de datos. Todos los coeficientes de correlación se encuentran muy próximos a cero, lo que indica asociaciones positivas o negativas extremadamente débil

Análisis de regresión lineal

```
In [183... from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X_independientes = columnas_cuantitativas.copy()
X_independientes.remove("FinalGrade")
X_independientes.remove("StudentID")
```

Modelo de regresión lineal simple

```
In [185... for var in X_independientes:
    x = df_estudiantes[var].values.reshape(-1,1)
    y = df_estudiantes["FinalGrade"]
    lm = LinearRegression()
    lm.fit(x,y)
    y_pred = lm.predict(x)
```

```
r2 = r2_score(y,y_pred)
print("El valor del r2 con la variable %s es %.5f" %(var, r2))
```

El valor del r2 con la variable AttendanceRate es 0.00020
El valor del r2 con la variable StudyHoursPerWeek es 0.00120
El valor del r2 con la variable PreviousGrade es 0.00001
El valor del r2 con la variable Study Hours es 0.00195
El valor del r2 con la variable Attendance (%) es 0.00126

Modelo de regresión múltiple

```
In [187... predictoras = df_estudiantes[X_independientes]
respuesta = df_estudiantes["FinalGrade"]
```

```
lm_multiple = LinearRegression()
lm_multiple.fit(predictoras, respuesta)
```

Out[187...

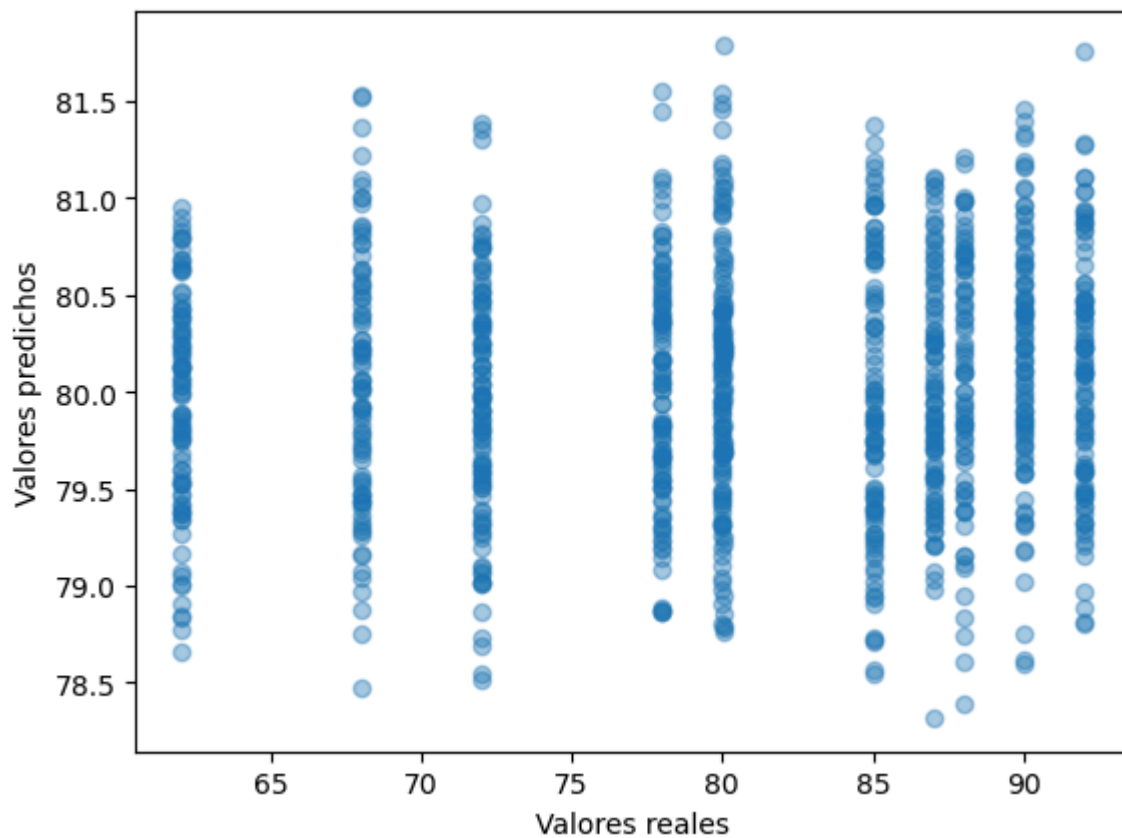
```
▼ LinearRegression ⓘ ?
LinearRegression()
```

```
In [189... y_pred = lm_multiple.predict(predictoras)
r2_multiple = r2_score(respuesta,y_pred)

print("El valor del r2 que se tiene para el modelo es %s" %r2_multiple)
```

El valor del r2 que se tiene para el modelo es 0.004578889208947001

```
In [191... plt.figure()
plt.scatter(respuesta, y_pred, alpha=0.4)
plt.xlabel("Valores reales")
plt.ylabel("Valores predichos")
plt.show()
```



In [193...

```
import statsmodels.api as sm
from scipy import stats

#Añadir el intercepto
X2 = sm.add_constant(predictoras)
#Ajustar el modelo
est = sm.OLS(respuesta, X2)

#Ver el ajuste
est2 = est.fit()
print(est2.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  FinalGrade    R-squared:                  0.005
Model:                          OLS          Adj. R-squared:          -0.000
Method:                         Least Squares  F-statistic:              0.9145
Date:                           Sat, 01 Aug 2026  Prob (F-statistic):      0.471
Time:                           10:17:09      Log-Likelihood:          -3646.3
No. Observations:                1000         AIC:                     7305.
Df Residuals:                    994         BIC:                     7334.
Df Model:                        5
Covariance Type:                 nonrobust
=====
=====
=====
coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const                78.0039      4.476      17.427      0.000      69.221      8
6.787
AttendanceRate       -0.0190      0.041      -0.463      0.643      -0.100
0.062
StudyHoursPerWeek     0.0515      0.048       1.067      0.286      -0.043
0.146
PreviousGrade         0.0043      0.030       0.144      0.885      -0.055
0.063
Study Hours           0.2930      0.208       1.409      0.159      -0.115
0.701
Attendance (%)         0.0221      0.021       1.072      0.284      -0.018
0.063
=====
Omnibus:                153.428    Durbin-Watson:              2.040
Prob(Omnibus):           0.000    Jarque-Bera (JB):          77.849
Skew:                    -0.528    Prob(JB):                  1.25e-17
Kurtosis:                2.132    Cond. No.                  2.13e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.13e+03. This might indicate that there are strong multicollinearity or other numerical problems.

El modelo de regresión lineal múltiple presenta una capacidad explicativa muy baja, con un $R^2=0.005$ y un R^2 ajustado aproximadamente igual a cero. Además, ninguna de las variables explicativas presentó un valor $p<0.05$.

Se recomienda revisar la calidad y definición de las variables, incorporar otros factores relevantes y explorar posibles relaciones no lineales o modelos alternativos.

In []: